

Bucket sort

Bucket sort, or **bin sort**, is a sorting algorithm that works by distributing the elements of an array into a number of buckets. Each bucket is then sorted individually, either using a different sorting algorithm, or by recursively applying the bucket sorting algorithm. It is a distribution sort, a generalization of pigeonhole sort that allows multiple keys per bucket, and is a cousin of radix sort in the most-to-least significant digit flavor. Bucket sort can be implemented with comparisons and therefore can also be considered a comparison sort algorithm. The computational complexity depends on the algorithm used to sort each bucket, the number of buckets to use, and whether the input is uniformly distributed.

Bucket sort

Class	<u>Sorting algorithm</u>
Data structure	<u>Array</u>
Worst-case performance	$O(n^2)$
Average performance	$O\left(n + \frac{n^2}{k} + k\right)$, where k is the number of buckets. $O(n)$, when $k \approx n$.
Worst-case space complexity	$O(n + k)$

Bucket sort works as follows:

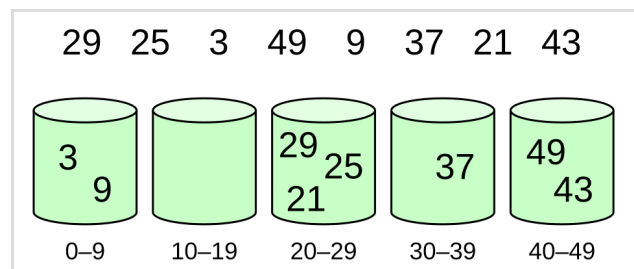
1. Set up an array of initially empty "buckets".
2. **Scatter**: Go over the original array, putting each object in its bucket.
3. Sort each non-empty bucket.
4. **Gather**: Visit the buckets in order and put all elements back into the original array.

Pseudocode

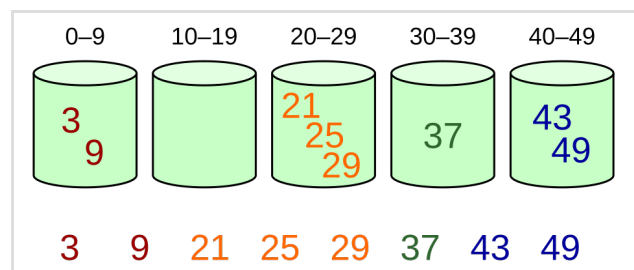
```

function bucketSort(array, k) is
    buckets ← new array of k empty lists
    M ← 1 + the maximum key value in the array
    for i = 0 to length(array) do
        insert array[i] into buckets[floor(k × array[i] / M)]
    for i = 0 to k do
        nextSort(buckets[i])
    return the concatenation of buckets[0], ..., buckets[k]
```

Let *array* denote the array to be sorted and *k* denote the number of buckets to use. One can compute the maximum key value in linear time by iterating over all the keys once. The floor function must be used to convert a floating number to an integer (and possibly casting of datatypes too). The function *nextSort* is a sorting function used to sort each bucket.



Elements are distributed among bins



Then, elements are sorted within each bin

Conventionally, insertion sort is used due to its relatively high performance with small numbers of elements, but other algorithms could be used as well, such as selection sort or merge sort. Using *bucketSort* itself as *nextSort* produces a relative of radix sort; in particular, the case $n = 2$ corresponds to quicksort (although potentially with poor pivot choices).

Analysis

Worst-case analysis

When the input contains several keys that are close to each other (clustering), those elements are likely to be placed in the same bucket, which results in some buckets containing more elements than average. The worst-case scenario occurs when all the elements are placed in a single bucket. The overall performance would then be dominated by the algorithm used to sort each bucket, for example $O(n^2)$ insertion sort or $O(n \log(n))$ comparison sort algorithms, such as merge sort.

Average-case analysis

Consider the case that the input is uniformly distributed. The first step, which is **initialize** the buckets and **find the maximum key value** in the array, can be done in $O(n)$ time. If division and multiplication can be done in constant time, then **scattering** each element to its bucket also costs $O(n)$. Assume insertion sort is used to sort each bucket, then the third step costs $O(\sum_{i=1}^k n_i^2)$, where n_i is the length of the bucket indexed i . Since we are concerning the average time, the expectation $E(n_i^2)$ has to be evaluated instead. Let X_{ij} be the random variable that is **1** if

element j is placed in bucket i , and **0** otherwise. We have $n_i = \sum_{j=1}^n X_{ij}$. Therefore,

$$\begin{aligned} E(n_i^2) &= E\left(\sum_{j=1}^n X_{ij} \sum_{l=1}^n X_{il}\right) \\ &= E\left(\sum_{j=1}^n \sum_{l=1}^n X_{ij} X_{il}\right) \\ &= E\left(\sum_{j=1}^n X_{ij}^2\right) + E\left(\sum_{1 \leq j, l \leq n} \sum_{j \neq l} X_{ij} X_{il}\right). \end{aligned}$$

The last line separates the summation into the case $j = l$ and the case $j \neq l$. Since the chance of an object distributed to bucket i is $1/k$, X_{ij} is 1 with probability $1/k$ and 0 otherwise.

$$E(X_{ij}^2) = 1^2 \cdot \left(\frac{1}{k}\right) + 0^2 \cdot \left(1 - \frac{1}{k}\right) = \frac{1}{k}$$

$$E(X_{ij}X_{ik}) = 1 \cdot \left(\frac{1}{k}\right) \left(\frac{1}{k}\right) = \frac{1}{k^2}$$

With the summation, it would be

$$E\left(\sum_{j=1}^n X_{ij}^2\right) + E\left(\sum_{1 \leq j, k \leq n} \sum_{j \neq k} X_{ij}X_{ik}\right) = n \cdot \frac{1}{k} + n(n-1) \cdot \frac{1}{k^2} = \frac{n^2 + nk - n}{k^2}$$

Finally, the complexity would be

$$O\left(\sum_{i=1}^k E(n_i^2)\right) = O\left(\sum_{i=1}^k \frac{n^2 + nk - n}{k^2}\right) = O\left(\frac{n^2}{k} + n\right).$$

The last step of bucket sort, which is **concatenating** all the sorted objects in each bucket, requires $O(k)$ time. Therefore, the total complexity is $O\left(n + \frac{n^2}{k} + k\right)$. Note that if k is chosen to be $k = \Theta(n)$, then bucket sort runs in $O(n)$ average time, given a uniformly distributed input.^[1]

Optimizations

A common optimization is to put the unsorted elements of the buckets back in the original array *first*, then run insertion sort over the complete array; because insertion sort's runtime is based on how far each element is from its final position, the number of comparisons remains relatively small, and the memory hierarchy is better exploited by storing the list contiguously in memory.^[2]

If the input distribution is known or can be estimated, buckets can often be chosen which contain constant density (rather than merely having constant size). This allows $O(n)$ average time complexity even without uniformly distributed input.

Variants

Generic bucket sort

The most common variant of bucket sort operates on a list of n numeric inputs between zero and some maximum value M and divides the value range into b buckets each of size M/b . If each bucket is sorted using insertion sort, the sort can be shown to run in expected linear time (where the average is taken over all possible inputs).^[3] However, the performance of this sort degrades with clustering; if many values occur close together, they will all fall into a single bucket and be sorted slowly. This performance degradation is avoided in the original bucket sort algorithm by assuming that the input is generated by a random process that distributes elements uniformly over the interval $[0, 1)$.^[1]

ProxmapSort

Similar to generic bucket sort as described above, **ProxmapSort** works by dividing an array of keys into subarrays via the use of a "map key" function that preserves a partial ordering on the keys; as each key is added to its subarray, insertion sort is used to keep that subarray sorted, resulting in the entire array being in sorted order when ProxmapSort completes. ProxmapSort differs from bucket sorts in its use of the map key to place the data approximately where it belongs in sorted order, producing a "proxmap" — a proximity mapping — of the keys.

Histogram sort

Another variant of bucket sort known as histogram sort or counting sort adds an initial pass that counts the number of elements that will fall into each bucket using a count array.^[4] Using this information, the array values can be arranged into a sequence of buckets in-place by a sequence of exchanges, leaving no space overhead for bucket storage.

Postman's sort

The **Postman's sort** is a variant of bucket sort that takes advantage of a hierarchical structure of elements, typically described by a set of attributes. This is the algorithm used by letter-sorting machines in post offices: mail is sorted first between domestic and international; then by state, province or territory; then by destination post office; then by routes, etc. Since keys are not compared against each other, sorting time is $O(cn)$, where c depends on the size of the key and number of buckets. This is similar to a radix sort that works "top down," or "most significant digit first."^{[5][6]}

Shuffle sort

The **shuffle sort**^[7] is a variant of bucket sort that begins by removing the first $1/8$ of the n items to be sorted, sorts them recursively, and puts them in an array. This creates $n/8$ "buckets" to which the remaining $7/8$ of the items are distributed. Each "bucket" is then sorted, and the "buckets" are concatenated into a sorted array.

Comparison with other sorting algorithms

Bucket sort can be seen as a generalization of counting sort; in fact, if each bucket has size 1 then bucket sort degenerates to counting sort. The variable bucket size of bucket sort allows it to use $O(n)$ memory instead of $O(M)$ memory, where M is the number of distinct values; in exchange, it gives up counting sort's $O(n + M)$ worst-case behavior.

Bucket sort with two buckets is effectively a version of quicksort where the pivot value is always selected to be the middle value of the value range. While this choice is effective for uniformly distributed inputs, other means of choosing the pivot in quicksort such as randomly selected pivots make it more resistant to clustering in the input distribution.

The n -way mergesort algorithm also begins by distributing the list into n sublists and sorting each one; however, the sublists created by mergesort have overlapping value ranges and so cannot be recombined by simple concatenation as in bucket sort. Instead, they must be interleaved by a merge algorithm. However, this added expense is counterbalanced by the simpler scatter phase and the ability to ensure that each sublist is the same size, providing a good worst-case time bound.

Top-down radix sort can be seen as a special case of bucket sort where both the range of values and the number of buckets is constrained to be a power of two. Consequently, each bucket's size is also a power of two, and the procedure can be applied recursively. This approach can accelerate the scatter phase, since we only need to examine a prefix of the bit representation of each element to determine its bucket.

References

1. Thomas H. Cormen; Charles E. Leiserson; Ronald L. Rivest & Clifford Stein. *Introduction to Algorithms*. "Bucket sort runs in linear time on the average. Like counting sort, bucket sort is fast because it assumes something about the input. Whereas counting sort assumes that the input consists of integers in a small range, bucket sort assumes that the input is generated by a random process that distributes elements uniformly over the interval $[0,1)$. The idea of bucket sort is to divide the interval $[0, 1)$ into n equal-sized subintervals, or buckets, and then distribute the n input numbers into the buckets. Since the inputs are uniformly distributed over $[0, 1)$, we don't expect many numbers to fall into each bucket. To produce the output, we simply sort the numbers in each bucket and then go through the buckets in order, listing the elements in each."
 2. Corwin, E. and Logar, A. "Sorting in linear time — variations on the bucket sort". *Journal of Computing Sciences in Colleges*, 20, 1, pp.197–202. October 2004.
 3. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 8.4: Bucket sort, pp.174–177.
 4. NIST's Dictionary of Algorithms and Data Structures: histogram sort (<https://xlinux.nist.gov/dads/HTML/histogramSort.html>)
 5. Black, Paul E., ed. (20 June 2011). "postman's sort" (<https://xlinux.nist.gov/dads/HTML/postmansort.html>). *Dictionary of Algorithms and Data Structures*. National Institute of Standards and Technology. Retrieved 31 March 2026. This article incorporates text from this source, which is in the public domain.
 6. Ramey, Robert (August 1992). "The Postman's Sort" (http://web.archive.org/web/20090628192742/https://rrsd.com/software_development/postmans_sort/cuj/cuj.htm). *C/C++ Users Journal*. Archived from the original (<http://www.rrsd.com/psort/cuj/cuj.htm>) on 28 June 2009.
 7. A revolutionary new sort from John Cohen Nov 26, 1997 (<https://groups.google.com/group/fido7.ru.algorithms/msg/26084cdb04008ab3>)
- NIST's Dictionary of Algorithms and Data Structures: bucket sort (<https://xlinux.nist.gov/dads/>)

[HTML/bucketsort.html](#))

External links

- [Bucket Sort Code for Ansi C \(http://www.dcc.uchile.cl/~rbaeza/handbook/algs/4/423.sort.c.html\)](http://www.dcc.uchile.cl/~rbaeza/handbook/algs/4/423.sort.c.html)
 - [Variant of Bucket Sort with Demo \(http://www1bpt.bridgeport.edu/~dichter/lilly/bucketsort.htm\)](http://www1bpt.bridgeport.edu/~dichter/lilly/bucketsort.htm)
-

Retrieved from "https://en.wikipedia.org/w/index.php?title=Bucket_sort&oldid=1346393165"